

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 07 -

USO DE SDKS E DESIGN DE APIS PARA PIPELINES ML

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	6
MERCADO, CASES E TENDÊNCIAS	19
O QUE VOCÊ VIU NESTA AULA?	24
REFERÊNCIAS.....	25

LEANDRO

O QUE VEM POR AÍ?

Imagine poder treinar e implantar um modelo de machine learning com poucas linhas de código, sem precisar clicar em inúmeras telas ou escrever scripts desconexos. Os SDKs (Kits de Desenvolvimento de Software) surgem justamente para isso: eles reúnem bibliotecas e ferramentas que expõem funcionalidades de uma plataforma de ML de forma programática, permitindo que cientistas de dados e engenheiros integrem facilmente tarefas de ML em aplicações e pipelines.

Por exemplo, em vez de usar um portal web para configurar um experimento de IA na nuvem, você pode usar um SDK como o do Azure ML ou AWS SageMaker para fazê-lo diretamente em Python, automatizando desde o preparo dos dados até o deploy de um serviço de inferência. Mas ter um SDK não basta – é preciso que suas APIs (Interfaces de Programação) sejam bem projetadas para serem fáceis de usar, consistentes e extensíveis. Se a API for confusa ou inconsistente, desenvolvedores(as) perdem tempo com erros, configurações repetitivas e código pouco reutilizável.

Nesta aula, vamos explorar como são construídos os SDKs de ML e os princípios de design de APIs que tornam bibliotecas como scikit-learn e frameworks de pipelines como Kubeflow tão eficazes. Também iremos refletir sobre casos de uso reais: quais problemas essas abordagens resolvem no dia a dia e como um bom design pode acelerar experimentos e evitar armadilhas comuns. Fique conosco – ao final, você entenderá melhor como usar e até criar SDKs/APIs que simplificam pipelines de ML e estará pronto(a) para aplicar esses conceitos nos desafios práticos do curso. Não deixe de assistir às videoaulas, que demonstram esses conceitos em ação em uma situação real!

HANDS ON

Nesta seção prática, vamos implementar um mini pipeline de Machine Learning usando Python – afinal, o Python é uma das linguagens mais utilizadas em ciência de dados e possui amplo suporte a SDKs e bibliotecas de ML. O objetivo do hands on é ilustrar como uma API bem desenhada ajuda a compor etapas de processamento de forma intuitiva. Prepare seu ambiente e acompanhe: nos vídeos, mostramos passo a passo a construção e execução desse pipeline diretamente em um Jupyter Notebook, reforçando a importância da consistência de interfaces. A seguir, apresentamos um extrato do código-fonte usado, com comentários explicativos:

```
# Código-fonte 1 – Exemplo de pipeline simples com scikit-learn. Fonte: Elaborado pelo autor (2025).
from sklearn.pipeline import Pipeline
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression

# Definimos um pipeline com duas etapas: redução de dimensionalidade (PCA) seguida de classificação (Regressão Logística)
pipeline = Pipeline([
    ('reduzir_dimensionalidade', PCA(n_components=2)),          # Transformador: PCA para reduzir dados a 2
    ('classificador', LogisticRegression())                   # Estimador final: regressão logística para
])                                                            classificação

## Suponha que X_train, y_train, X_test sejam nossos dados de treino e teste.
pipeline.fit(X_train, y_train)    # Treina sequencialmente: primeiro o PCA, depois a regressão nos dados
                                  transformados.
```

Código-fonte 1 – Código-fonte com comentários explicativos
Fonte: Elaborado pelo autor (2026)

No código apresentado, usando a API unificada do scikit-learn, encadeamos duas operações em um único objeto Pipeline. Os métodos familiares `.fit()` e `.predict()` funcionam de ponta a ponta: ao chamar `fit`, o PCA é ajustado e em seguida a regressão é treinada com os dados transformados; ao chamar `predict`, os dados de teste passam automaticamente pelo PCA antes da predição. Perceba como a interface coesa do scikit-learn facilita o fluxo – não precisamos manualmente aplicar cada passo separadamente, pois a API do pipeline abstrai essa sequência de forma clara.

Siga as instruções das videoaulas para reproduzir este exemplo no seu computador, experimente alterar parâmetros (por exemplo, o número de componentes do PCA ou o tipo de classificador) e observe como, graças ao design consistente da API, é simples testar variações. Essa prática guiada visa consolidar seu entendimento: ao final do hands on, pause o vídeo e tente você mesmo(a) criar um pequeno pipeline ou usar um SDK de sua escolha em um miniprojeto. Essa experiência prática é fundamental.



SAIBA MAIS

Agora que vimos uma amostra prática, vamos nos aprofundar nos conceitos. Começaremos definindo formalmente o que é um SDK em ML e por que ele é útil. Em seguida, investigaremos os princípios de design de APIs que tornam bibliotecas de ML produtivas, citando exemplos notáveis (scikit-learn, Keras, TensorFlow/Kubeflow). Por fim, discutiremos questões avançadas – desde impactos de design na performance (CPU/GPU, memória) até extensibilidade e manutenção de pipelines em projetos de ciência de dados.

SDKs de Machine Learning – O que são e para que servem?

SDK (Software Development Kit), ou Kit de Desenvolvimento de Software, refere-se a um conjunto de ferramentas e bibliotecas fornecido por uma plataforma, sistema ou framework para auxiliar desenvolvedores(as) a criarem aplicações para aquela plataforma. No contexto de machine learning, um SDK normalmente inclui bibliotecas (APIs) específicas para acessar serviços de ML (treinamento, deploy etc.), utilitários de autenticação, ferramentas de linha de comando, documentação e, às vezes, ambientes de runtime ou exemplos. A ideia é oferecer todos os componentes necessários para que o usuário consiga interagir com a plataforma de forma programática, escrevendo código em vez de usar interfaces manuais.

Por exemplo, o Azure Machine Learning SDK (da Microsoft) permite ao desenvolvedor Python gerenciar todo o ciclo de vida de experimentos de ML na nuvem diretamente do código. Com ele, podemos: registrar e versionar conjuntos de dados, preparar e enviar scripts de treinamento para execução em clusters de GPU, monitorar métricas de execução e até abrir um serviço web com o modelo treinado – tudo via chamadas de função, sem sair do notebook.

Similarmente, o AWS SageMaker SDK oferece objetos e métodos para configurar trabalhos de treinamento no SageMaker, criar endpoints de inferência escaláveis etc., sem precisar passar pelo console web da AWS. Esses SDKs normalmente incluem também wrappers para frameworks populares (como TensorFlow, PyTorch, scikit-learn), facilitando a integração – por exemplo, o SageMaker SDK tem classes como TensorFlowEstimator ou SKLearnEstimator que

automatizam o empacotamento do código do usuário dentro do ambiente gerenciado do serviço.

Em suma, um SDK de ML atua como ponte entre o indivíduo desenvolvedor e a plataforma de ML, abstraindo detalhes de infraestrutura e provendo uma experiência de alto nível. Isso acelera o desenvolvimento (menos código “cerimonial” repetitivo) e reduz erros, pois o kit tende a vir com padrões prontos e boas práticas incorporadas. Contudo, para colher esses benefícios, o SDK precisa ser bem projetado – e é aí que entra a discussão de design de APIs.

Vale destacar a diferença: a API (Application Programming Interface) é a interface (conjunto de classes, funções, métodos etc.) que o(a) desenvolvedor(a) utiliza. Já o SDK geralmente inclui uma API (ou várias) dentro de um pacote mais amplo. Por exemplo, o SDK do Azure ML contém a API Python do Azure ML (com classes como Workspace, Experiment, Model etc.), além de ferramentas auxiliares (CLI, exemplos, documentação). Todo SDK normalmente expõe uma ou mais APIs para consumo. Dito isso, no contexto desta aula, muitas vezes falaremos de “API do SDK” ou apenas “API de ML” de forma intercambiável, referindo-nos às interfaces concretas que o indivíduo usa ao programar funcionalidades de ML.

Princípios de design de APIs em bibliotecas de ML

Por que algumas bibliotecas de ML são amadas e outras causam frustração? A resposta frequentemente está no design da API. Uma API bem desenhada faz até tarefas complexas parecerem simples, enquanto um design descuidado pode tornar árduo até o básico. Vamos examinar alguns dos principais princípios de design de APIs, com ênfase especial em bibliotecas de machine learning. Esses princípios foram destacados tanto pela comunidade acadêmica (e.g., Buitinck et al., 2013, ao documentar o design do scikit-learn) quanto por praticantes experientes (por exemplo, François Chollet, criador do Keras, enfatizando o “UX da API” em seu blog). A seguir, compilamos diretrizes universais, ilustrando cada uma com exemplos.

Consistência e uniformidade: todos os componentes de uma biblioteca devem seguir convenções similares – mesmos nomes para operações equivalentes, mesma ordem de parâmetros, comportamento previsível. A equipe do scikit-learn, por exemplo, adotou uma interface uniforme para todos os algoritmos: instanciar o objeto,

chamar `fit(X, y)` para treinar e depois `predict(X)` ou `transform(X)` para usar o modelo. Seja um modelo de regressão linear ou uma floresta aleatória, a forma de uso é a mesma. Essa consistência traz uma enorme vantagem: baixo esforço cognitivo para o usuário aprender novas funcionalidades.

Se você já usou um classificador no scikit-learn, consegue usar qualquer outro seguindo o mesmo padrão – trocar um `LogisticRegression` por um `SVC` (máquina de vetores de suporte) requer mudar apenas a classe, todo o resto do código permanece igual. Bibliotecas coesas também mantêm consistência com o ecossistema: por exemplo, o scikit-learn utiliza tipos de dados do NumPy (arrays) em vez de inventar seu próprio objeto de matriz, garantindo familiaridade e fácil integração.

Simplicidade e mínimos necessários: boas APIs evitam exigir do usuário configurações ou passos desnecessários. Deve-se tentar oferecer valores default sensatos para hiperparâmetros e opções, de modo que o uso básico funcione com o mínimo de código. Novamente o scikit-learn se destaca – praticamente todos os parâmetros de modelos têm um default razoável (e.g., `LogisticRegression()` já vem com penalidade L2 e outras configs padrão úteis). Assim, um usuário iniciante pode simplesmente chamar `clf = LogisticRegression(); clf.fit(X, y)` e obter um resultado viável, enquanto um usuário avançado ainda tem a opção de ajustar tudo se precisar. Essa filosofia de “opiniated defaults” (padrões inteligentes) agiliza experimentos e reduz a barreira de entrada para não-especialistas.

Outro aspecto de simplicidade é: não sobrecarregar a interface com conceitos novos. Cada novo objeto ou abstração que o usuário precise aprender adiciona carga cognitiva. Muitas vezes, é melhor reaproveitar conceitos já existentes. Exemplo: no Keras (biblioteca de redes neurais de alto nível), quase tudo gira em torno de dois conceitos principais – camadas (`Layer`) e modelos (`Model`) – e operações se expressam compondo esses blocos. Em vez de expor cinco tipos diferentes de “modelos” para casos distintos, o Keras unifica tudo no mesmo conceito de modelo sequencial ou funcional. Menos conceitos para o usuário dominar significa aprendizagem mais rápida.

Fluxos intuitivos e foco no usuário: uma API deve ser pensada do ponto de vista do usuário, não espelhar detalhes intrincados da implementação interna. Isso requer design centrado na experiência do desenvolvedor – entender como as pessoas vão utilizar a ferramenta em cenários reais e otimizar para que esses cenários sejam

o mais diretos possível. François Chollet (2017) defende projetar workflows de uso end-to-end deliberadamente. Ou seja, ao criar uma API de ML, os indivíduos desenvolvedores deveriam listar as tarefas comuns (e.g., “treinar modelo X e validá-lo com técnica Y”, “carregar dados, extrair features, aplicar algoritmo, avaliar resultado”) e garantir que a sequência de chamadas para realizar essas tarefas seja fluida e sem “gambiarras”.

Em bibliotecas bem planejadas, as etapas do usuário correspondem de perto aos conceitos do domínio do problema – por exemplo, na API de um framework de Deep Learning, faz sentido haver objetos como Camada, Otimizador, Época, pois esses são termos que o usuário normalmente pensa. Em contrapartida, não se deve obrigar o usuário a lidar com objetos que fazem sentido só internamente (como um GraphExecutor ou SessionManager exposto sem necessidade, algo que aconteceu com versões iniciais do TensorFlow e gerou duras críticas).

Uma forma simples de avaliar se o fluxo é intuitivo é: quantas chamadas ou quanto código “cerimonial” é necessário para realizar uma tarefa objetiva? Se para treinar e avaliar um modelo simples a biblioteca exigir 20 etapas/configurações, talvez o design esteja pedindo ajuste. Por outro lado, se é possível realizar em 3-4 linhas por meio de abstrações bem escolhidas, provavelmente a UX está boa. A pipeline API do Hugging Face Transformers é um exemplo moderno: para fazer sentiment analysis em uma frase, basta: `from transformers import pipeline; sentiment = pipeline("sentiment-analysis"); sentiment("Texto de exemplo")` – internamente isso carrega um modelo pré-treinado, tokeniza o texto, processa e retorna o resultado, mas o usuário não precisa se preocupar com esses detalhes debaixo do capô. A API foca no que o usuário quer fazer (analisar sentimento), não em como exatamente fazer (carregar pesos, executar tensores etc.).

Consistência de nomenclatura e estilo de código: em complemento à consistência funcional, a consistência estética e de nomenclatura é fundamental. Pequenas variações de nome podem confundir e gerar erros sutis. Se uma biblioteca usa `num_epochs` em um lugar, não deveria usar `n_epochs` em outro – escolha uma convenção e aplique em todos os métodos. Além disso, aderir a convenções consagradas do ecossistema evita frustrações. Um caso citado por Chollet (2017) é o uso de `axis` vs `dim` como nome de parâmetro: o NumPy usa `axis` para se referir a eixos de arrays; se sua API de ML lida com tensores, seguir `axis` para manter familiaridade

é indicado, em vez de introduzir do nada um nome `dim` ou `dimension` para a mesma coisa.

Padrões de nomenclatura coesos facilitam o autocompletar e a memorização – desenvolvedores(as) rapidamente internalizam “objeto.alguma_coisa_” e o IDE já sugere as opções esperadas. Documentação e mensagens de erro entram aqui também: métodos claramente documentados, com exemplos e erros que fazem sentido em vez de códigos obscuros, melhoram drasticamente a usabilidade da API. Idealmente, uma boa API chega ao ponto em que o usuário consegue adivinhar o que fazer sem consultar documentação a todo instante, pois as opções “fazem sentido”. Um teste ácido proposto por Chollet: se um usuário conseguir lembrar de como usar a API em um cenário comum no dia seguinte após ter lido o doc/tutorial apenas uma vez, então a API tem uma baixa carga cognitiva e provavelmente foi bem desenhada.

Extensibilidade e composição: APIs de sucesso geralmente são concebidas como block-building – provêm peças que a pessoa desenvolvedora possa combinar ou estender para alcançar diferentes objetivos. Isso é crucial em Machine Learning, porque cada projeto pode exigir uma variante ligeiramente diferente. Se a API é muito rígida, o usuário vai “bater no teto” e possivelmente abandonar a biblioteca. Por outro lado, se for maleável, ele consegue adaptar a ferramenta ao problema.

No scikit-learn, além do core consistente, existe a possibilidade de o usuário criar seu próprio estimador seguindo as convenções (implementar métodos `fit`, `predict`, etc. em uma classe) e então usá-lo dentro do framework como se fosse integrado. Isso é possível graças ao princípio do duck typing (se anda como um pato e grasna como um pato, então é um pato) – ou seja, qualquer classe que implemente a interface esperada do scikit-learn é tratada como parte do ecossistema sem necessidade de heranças complexas.

Além disso, o scikit-learn encoraja a composição: fluxos de ML muitas vezes envolvem sequência de transformações nos dados (normalização, redução de dimensionalidade etc.) seguidas de um modelo final. Em vez de exigir código “procedural” fazendo cada passo, a biblioteca oferece o objeto `Pipeline` (que utilizamos no hands on) e também o `FeatureUnion` (que combina saídas paralelas de transformadores), permitindo compor facilmente etapas modulares. Por exemplo, você pode compor um pipeline com um `StandardScaler` -> `PCA` -> `SVM` e tratá-lo como uma única unidade (chamar `fit` treina tudo na ordem; um `GridSearch` pode ajustar

hiperparâmetros de qualquer passo interno). Essa capacidade de composição aumenta o poder expressivo da API sem multiplicar complexidade – pequenos blocos interoperáveis geram muitas possibilidades.

Outra faceta de extensibilidade: permitir plug-ins ou customização de comportamentos. Frameworks de deep learning, por exemplo, expõem mecanismos para que usuários definam suas próprias funções de perda, métricas ou até camadas customizadas de forma relativamente simples, em vez de ficarem limitados ao que vem pronto. Claro, há um balanço: expor muitos pontos de extensão aumenta a superfície da API (que deve ser mantida, testada, documentada). Portanto, um bom design define extensões onde realmente importa para o usuário especializado, mas mantém o uso padrão simples para o usuário casual.

Feedback claro e útil: embora não seja “design da API” em si, a experiência do usuário é altamente influenciada pelas mensagens de erro e aviso que o sistema fornece. APIs bem projetadas antecipam erros comuns e fornecem mensagens explicativas, eventualmente com sugestões de solução. Por exemplo, o Keras ficou conhecido por emitir erros legíveis – se o usuário passar dados de forma errada, a mensagem pode explicar exatamente qual era o formato esperado e como converter o dado, em vez de dar um stacktrace genérico. Esse cuidado com feedback torna a curva de aprendizado menos íngreme.

Um “design” ruim aqui pode arruinar os outros acertos – imagine uma biblioteca perfeita no papel, mas que quando algo dá errado solta “Exception: Error 1729” – o(a) desenvolvedor(a) fica sem saber o que fazer. Portanto, do ponto de vista de engenharia de APIs, incluir validações de entrada (para detectar uso incorreto o mais cedo possível) e incluir mensagens úteis faz parte de um bom design orientado ao usuário.

Em resumo, projetar uma API de ML é tanto uma arte quanto uma engenharia. É preciso se colocar no lugar do usuário (muitas vezes, você mesmo algum tempo atrás) e garantir que a interface seja consistente, simples nos casos comuns, flexível nos casos especiais e que guie o indivíduo desenvolvedor na direção certa. Quando esses princípios são seguidos, a API “desaparece” – isto é, o usuário consegue concentrar-se no problema de ciência de dados em si, em vez de lutar contra a ferramenta. E, implicitamente, isso leva a um código mais correto e soluções mais rápidas. Como veremos a seguir, bibliotecas consagradas adotaram essas ideias e

obtiveram grande sucesso, enquanto projetos que ignoraram a experiência do(a) desenvolvedor(a) acabaram sendo substituídos por alternativas mais amigáveis.

Exemplos de APIs bem-sucedidas em ML

Scikit-learn – já citado extensivamente – é um caso emblemático de sucesso de design de API. Lançado em 2010 e crescendo ao longo da década, esse pacote fez tanto sucesso não por ter os algoritmos mais inovadores (em sua maioria, ele implementa técnicas “clássicas” já estabelecidas), mas sim por torná-los acessíveis e combináveis de forma sem precedentes. Buitinck et al. (2013) relatam que a filosofia central do scikit-learn era ser “simples e eficiente, acessível a não-especialistas e reutilizável em diversos contextos”, evitando criar uma linguagem específica de ML e, em vez disso, aderindo ao “jeito Python” de fazer as coisas.

A interface básica compartilhada por todos os estimadores (as três interfaces: estimador para treino – fit; preditor – predict; e transformador – transform para pré-processadores) provou ser muito poderosa em termos de reusabilidade e composição. Por exemplo, graças a essa padronização, implementar um novo algoritmo e disponibilizá-lo no scikit-learn requer criar uma classe Python com métodos fit/predict adequados – imediatamente, ela se torna compatível com todo o ecossistema: pode ser usada em pipelines, em busca de hiperparâmetros (GridSearchCV) etc., sem código adicional.

Essa extensibilidade via duck typing incentivou até terceiros a lançarem pacotes complementares seguindo o mesmo estilo (ex.: sklearn-contrib com implementações experimentais; bibliotecas de terceiros como Imbalanced-learn e astroML adotam a interface sklearn para integração transparente). Outro ponto forte é a documentação exemplar, cheia de tutoriais e exemplos práticos, que ajudou a comunidade a aprender as melhores práticas juntamente com a API. Em suma, o scikit-learn virou sinônimo de API coesa a ponto de influenciar outras ferramentas.

Keras (e, por extensão, interfaces de alto nível em frameworks de Deep Learning) é outro exemplo notável. Lançado em 2015, inicialmente como front-end que podia rodar tanto em TensorFlow quanto Theano, o Keras focou em facilidade de uso e rapidez de prototipação. Seu autor definiu princípios como: user experience comes first, encourage progressive disclosure of complexity (ou seja, permitir usar

primeiro de forma simples e ir destravando controle maior conforme necessário) e provide consistent & simple APIs. A popularidade explosiva do Keras se deve a reduzir drasticamente a quantidade de código e conhecimento necessários para pôr de pé uma rede neural básica.

Por exemplo, treinar uma rede de classificação de imagens em TensorFlow “puro” (antes da integração do Keras) envolvia construir um grafo estático, gerenciar sessão, placeholders etc., enquanto em Keras resumiu-se a: definir modelo = Sequential([...]), compilar com model.compile(optimizer, loss) e depois model.fit(data, labels, epochs=...). Internamente, ainda aconteciam coisas complexas, mas invisíveis ao usuário comum. Essa abordagem levou a comunidade de pesquisa a adotar Keras massivamente – um código de exemplo de paper em Keras tendia a ser muito mais conciso e legível do que em frameworks de baixo nível. Tanto que o TensorFlow “absorveu” o Keras: a partir do TF 2.x, o Keras é a API de alto nível padrão do TensorFlow.

Isso ilustra um ponto importante: abstrações bem projetadas sobrevivem e viram padrão de mercado. Hoje praticamente todos os frameworks de deep learning oferecem uma API estilo Keras ou compatível. O PyTorch, que por sua vez se popularizou oferecendo uma experiência mais “pythonica” (execução dinâmica, define-by-run) e uma sintaxe muito próxima de NumPy, acabou adicionando também conveniências de alto nível (como torch.nn.Module e a abordagem de Lightning e FastAI para treinamento mais simples) para competir em facilidade de uso com o Keras. Em resumo, bibliotecas que empoderam sem complicar são adotadas mais rapidamente.

Um caso interessante é o das pipelines de ML em ambientes de produção. Ferramentas como TensorFlow Extended (TFX) e Kubeflow Pipelines (KFP) surgiram para ajudar a orquestrar workflows completos de ML (ingestão de dados -> treinamento -> validação -> deploy), muitas vezes em contextos de MLOps. Aqui o desafio de API é criar uma representação de pipeline que seja expressiva o bastante para descrever etapas diversas (incluindo executá-las distribuído, em contêineres isolados), mas ainda intuitiva para desenvolvedores(as) que não são especialistas em pipelines. O Kubeflow Pipelines optou por uma abordagem de DSL internal em Python: o usuário escreve um script Python usando construções decorativas para definir

componentes (funções Python anotadas que viram passos executáveis) e um pipeline que encadeia esses componentes.

Por trás dos panos, o KFP gera especificações em YAML/JSON (interpretação pelo sistema Argo Workflows no Kubernetes), mas o usuário não precisa escrever YAML manualmente – ele programa tudo em Python nativo. Isso é poderoso, pois aproveita a familiaridade com o Python e possibilita lógica dinâmica na definição do pipeline (loops, condicionais) usando a própria linguagem. Um exemplo simples: usando o SDK do KubeFlow, podemos definir duas funções Python, digamos `preprocess()` e `train_model()`, decorá-las como componentes do pipeline e depois definir uma função pipeline que chame `preprocess()` e passe sua saída para `train_model()`. Ao compilar esse pipeline, temos um workflow portátil, que pode ser executado na plataforma KubeFlow com um comando.

A API do KubeFlow Pipelines busca ser clara: os componentes são definidos como funções e a passagem de dados entre etapas aparece como chamadas de função encadeadas, o que é próximo do mental model do programador. Esse caso demonstra como APIs domain-specific podem ser criadas dentro de linguagens comuns para ofertar alto nível de abstração sem perder flexibilidade.

O TFX (do Google) segue filosofia similar, embora usando um DSL em Python mais configuracional para definir pipelines que rodam em ambientes como Apache Beam. Em todos esses casos, o objetivo é facilitar ao máximo a tarefa de orquestração: desenvolvedores(as) de ML querem especificar o que fazer em cada etapa e a ferramenta cuida de como/onde executar (local ou cloud, sequenciamento, paralelismo, manejo de dados). A escolha de Python como linguagem base desses SDKs não é por acaso: a comunidade de data science vive em Python, então uma API Python bem feita reduz imensamente a curva de aprendizado. Imaginem se KubeFlow exigisse escrever pipelines em XML ou Java – a adoção certamente seria bem menor!

Hugging Face e sua API Transformers merecem menção como exemplo contemporâneo de API de alto nível focada em produtividade. A função `pipeline()` dessa biblioteca, citada antes, permite ao usuário final aproveitar modelos de última geração pré-treinados para tarefas de NLP com literalmente uma linha de código. Essa é uma API de altíssimo nível – quase “mágica” – que encapsula uma série de etapas padronizadas (download do modelo, tokenização, inferência, decodificação) atrás de uma interface super simples. É claro que, para maior controle, a biblioteca

também expõe níveis mais baixos (você pode carregar manualmente um `AutoModel`, um `AutoTokenizer` e assim por diante e customizar).

Essa estratégia de múltiplos níveis de abstração é válida. O API de pipeline serve a quem quer rapidez e não precisa controlar detalhes; as APIs de modelo servem aos que precisam de ajustes finos. O importante é que uma camada não compromete a outra – usuários avançados não ficam limitados, enquanto iniciantes não ficam perdidos. Ter camadas claras na API é também um princípio de design (que o Hugging Face executa bem).

Para contrabalançar, vale citar brevemente APIs que não caíram no gosto e entender o porquê. Um caso conhecido foi o do TensorFlow 1.x (antes da reformulação do 2.x): apesar de ser extremamente poderoso e de alto desempenho, muitos usuários reclamavam da complexidade de sua API em comparação ao concorrente PyTorch. Na época (2016-2018), o TensorFlow exigia construir um grafo computacional estaticamente, iniciar uma sessão e rodar operadores sob um paradigma que não se integrava bem ao modo interativo do Python – o famoso `graph mode`. A API expunha conceitos como `tf.Session`, `tf.placeholder`, `tf.global_variables_initializer()`, etc., que soavam estranhos para quem só queria “treinar minha rede e obter resultados”.

Por outro lado, o PyTorch trouxe uma API em que executar uma operação era tão simples quanto chamar uma função Python e obter um resultado imediato (modo imperativo/eager). Essa diferença de filosofia de API (PyTorch priorizando a experiência do indivíduo desenvolvedor, TF priorizando performance via grafo) levou a uma “fuga” de pesquisadores(as) para o PyTorch por volta de 2017-2018. Mesmo com o TF oferecendo mais ferramentas de produção, a comunidade valorizou a simplicidade e intuitividade do PyTorch durante a fase de pesquisa/prototipagem – tanto que o TensorFlow teve que correr atrás e convergir suas APIs para algo semelhante ao Keras/PyTorch no TF2.

Essa história real ilustra como um design de API pode determinar o sucesso de uma plataforma. A lição aprendida foi que desempenho teórico não adianta se a usabilidade for baixa: desenvolvedores(as) preferem uma API que os deixe iterar rápido (mesmo que seja 5% menos otimizada) do que uma API que extrai cada gota de performance mas triplica o tempo de desenvolvimento ou depuração. Veremos mais sobre esse “caso de mercado” adiante.

Desempenho, recursos e melhores práticas de engenharia nas APIs de ML

Ao projetar (ou usar) um SDK/API de ML, não podemos esquecer dos aspectos de performance e engenharia de software, pois como a API é construída pode impactar tanto a eficiência do código gerado quanto a facilidade de manutenção a longo prazo. Felizmente, bons princípios de design muitas vezes andam de mãos dadas com bom desempenho. Vamos discutir alguns pontos relevantes.

Uso eficiente de CPU/GPU e paralelismo: uma preocupação comum em ML é aproveitar o hardware da melhor forma. Curiosamente, um design de API bem pensado pode ajudar a evitar padrões de uso ineficientes. Por exemplo, o scikit-learn optou por uma interface orientada a processamento em lote (batch) – métodos como `predict` esperam múltiplas amostras e retornam resultados em vetor – desencorajando o usuário de fazer loop item a item (o que seria terrivelmente lento em Python). Ao forçar a entrada ser um array de dados, a API de certa forma guia o(a) desenvolvedor(a) a utilizar vetorização, tirando proveito das implementações em C/C++ por baixo.

Ou seja, o design da API foi feito de olho em não “deixar brechas” para um uso que degrade demais a performance. Da mesma forma, frameworks de deep learning geralmente pedem tensores 4D (batch, altura, largura, canais) para entrada de redes neurais de visão, incentivando que se processe um minibatch por vez, o que casa bem com as otimizações de GPU. Além disso, algumas bibliotecas expõem parâmetros de paralelismo de forma simples, como scikit-learn usando `n_jobs` em várias funções para permitir usar múltiplos núcleos facilmente (internamente, ele cuida de threading ou multiprocessing).

Gerenciamento de memória: em pipelines de ML, é comum lidar com grandes volumes de dados. Uma API deve considerar isso – por exemplo, ser capaz de trabalhar com dados “streaming” ou por partes se necessário. O padrão iterator é uma maneira: bibliotecas como Gensim (de topic modeling) optaram por uma API baseada em iteradores Python para poder processar datasets enormes sem carregá-los inteiros na memória.

O scikit-learn, como discutido, adotou o modelo batch e, em alguns casos, oferece implementações online (incremental) para certos algoritmos, porém assumindo dados cabendo em memória para muitos casos, o que é uma escolha de

escopo (eles optaram por focar em usabilidade vs. big data; se for big data, talvez usar Spark MLlib ou Gensim seja mais apropriado – cada ferramenta com sua API alvo).

O importante aqui é que a API deixe claro o modelo de dados esperado e possibilite controlar o consumo de memória quando aplicável. Um exemplo: o TensorFlow oferece tanto o modo eager quanto a construção de datasets (via `tf.data.Dataset`) que pode fazer leitura sob demanda de arquivos etc. – a API de Dataset é uma minilinguagem para descrever pipelines de dados (mapeamentos, buffers, prefetching) que ajuda a otimizar a alimentação da GPU sem estourar RAM. Essa parte do design lida muito com pipelines de dados e vem se tornando crucial em bibliotecas modernas. Para o indivíduo desenvolvedor que utiliza, conhecer essas possibilidades (e.g., `prefetch`, `batch`, `shuffle` no `tf.data` API) ajuda a evitar gargalos de performance.

Controle de recursos externos: em plataformas como Azure ML ou Kubeflow, a API em Python interage com recursos de nuvem (clusters, nós, armazenamento). Uma boa API facilita monitorar e gerenciar esses recursos. Por exemplo, a classe `Run` do AzureML SDK permite consultar métricas e até cancelar execuções, tudo programaticamente. Boas práticas aqui incluem fornecer timeouts, métodos de cleanup (limpeza) e idempotência de operações quando possível – coisas importantes em engenharia de software robusta.

Do lado de quem projeta, esconder complexidade de rede e assegurar que chamadas bloqueantes sejam mínimas melhora a experiência (ninguém quer sua thread principal congelada porque a API está esperando resposta de servidor – a não ser que explicitamente documentado). Algumas APIs utilizam chamadas assíncronas ou retornos tipo `futures/promises` quando convém, mas isso deve ser feito com cuidado para não complicar o modelo mental do usuário.

Versionamento e manutenção: com o tempo, APIs evoluem. Aí entra uma questão de engenharia: como introduzir melhorias sem quebrar código existente? Frameworks de ML de longa vida (scikit-learn, TensorFlow, PyTorch) lidam com isso lançando versões maiores com deprecation warnings antes de mudar algo drástico e mantendo compatibilidade por um tempo. Do ponto de vista do usuário da API, é importante acompanhar as notas de versão e atualizar chamadas antigas.

Para minimizar dores, muitos SDKs seguem semântica estável no núcleo da API – ex.: o scikit até hoje mantém `fit/predict` – e, quando há grandes mudanças (e.g. TensorFlow 1 -> 2), costumam fornecer scripts de migração automatizada. Este curso não foca em manutenção de software, mas é bom ter ciência: APIs vivem em ecossistema. Sempre confira a documentação oficial e recomendações de versão para assegurar que você está usando práticas atualizadas e sustentáveis.

Melhores práticas de software na implementação: curiosamente, princípios de código limpo e modular também se aplicam dentro da construção de um SDK. Se um dia você estiver do lado de projetar internamente uma biblioteca de ML, se atente para separar bem camadas (por exemplo, separar a lógica de algoritmo da apresentação da API pública, assim você pode mudar otimizações internas sem afetar o usuário).

Inclusive, isso facilita controlar performance: muitas bibliotecas têm parte escrita em C/C++ otimizado e expõem a API em Python por cima – contanto que a fronteira esteja clara, essa divisão funciona bem. Também pense em testes: APIs consistentes são mais testáveis e frameworks de ML costumam ter extensas suítes de teste para garantir reprodutibilidade (afinal, erros silenciosos em resultados de ML podem ser catastróficos).

Resumindo esta seção teórica: o design de uma API de ML de sucesso combina cuidado com o indivíduo desenvolvedor (usabilidade) e com a eficiência computacional (performance). Os exemplos do mundo real mostraram que esses objetivos não são excludentes – muitas vezes, uma solução elegante resolve ambos. Um bom design abstrai detalhes para simplificar o uso, mas sem sacrificar a possibilidade de ajuste fino quando necessário. E ao usar essas ferramentas, nós, desenvolvedores(as), colhemos os frutos: escrevemos menos código, de forma mais clara e ainda assim rodamos modelos de modo rápido e confiável.

MERCADO, CASES E TENDÊNCIAS

Como nossas discussões são bem técnicas, é útil olhar para o **mundo real** e ver como SDKs e design de APIs influenciam projetos de Machine Learning em empresas e na comunidade. Nesta seção, vamos explorar alguns casos ilustrativos – tanto sucessos quanto lições aprendidas – e tendências atuais relacionadas ao uso de SDKs e APIs de ML.

Caso 1: A rivalidade TensorFlow vs. PyTorch – UX como fator decisivo

Nos anos 2010, o Google TensorFlow era a principal plataforma de deep learning, mas por volta de 2017 surgiu um fenômeno interessante: pesquisadores migrando em massa para o Facebook PyTorch. Por quê? Como já adiantamos, a experiência de desenvolvimento foi crucial. Comentários em fóruns e artigos da época descreviam o TensorFlow 1.x como “excessivamente complexo, cheio de sessões e scopes, parecia um Frankenstein de engenharia do Google”, enquanto elogiavam o PyTorch por “ser apenas Python e Numpy de forma intuitiva”. Em outras palavras, o design de API mais simples e direto do PyTorch conquistou a comunidade de P&D.

A consequência mercadológica foi enorme: hoje (2025), o PyTorch domina pesquisas acadêmicas e também ganhou espaço na indústria, forçando o TensorFlow a se reinventar (o TensorFlow 2 adotou o Keras e o eager execution, alinhando-se às expectativas dos usuários). Esse caso mostra que, mesmo em tecnologias de ponta, a ferramenta mais poderosa pode perder terreno se não for a mais utilizável. Empresas precisam ficar atentas: ao adotar um framework de ML, vale avaliar a curva de aprendizado para a equipe. E para quem desenvolve produtos de ML, fica a lição: invista em boa UX de desenvolvedor, pois ela pode determinar adoção ou abandono.

Caso 2: Michelangelo e o SDK interno do Uber

A Uber desenvolveu uma plataforma interna de ML chamada Michelangelo para atender suas necessidades de escala (previsão de ETAs, detecção de fraude etc.). Originalmente, o Michelangelo focou em performance e oferecia uma interface web e DSL próprias. Contudo, os cientistas de dados enfrentavam fricção ao tentar

experimentar novos algoritmos não suportados ou fluxos fora do padrão. Em 2018, o Uber anunciou o Michelangelo PyML, uma camada Python por cima de seu sistema que trouxe muito mais flexibilidade. Com o PyML, os usuários podiam usar código Python praticamente normal (incluindo bibliotecas como scikit-learn, pandas etc.) e, através de um SDK, implantar seus modelos no Michelangelo quase automaticamente.

A chave foi reconhecer que os desenvolvedores e desenvolvedoras preferem trabalhar em Python e que cada passo extra para portar modelos era um entrave. O PyML introduziu convenções simples: o cientista de dados implementa duas funções (`load_model` e `predict`) em uma classe modelo Python e registra no SDK; dali, a plataforma cuida do resto (versionar, criar contêiner Docker, distribuir em produção e até escalar batch predictions). Em outras palavras, o Uber criou um SDK interno para democratizar seu ML. O resultado foi agilizar o ciclo de inovação, permitindo que novos modelos saíssem do Jupyter para produção rapidamente, sem reescrita manual onerosa.

Esse caso evidencia a importância de APIs bem projetadas para MLOps: organizações que criam frameworks internos devem priorizar a usabilidade para seus times, caso contrário arriscam baixa adoção interna (no pior caso, equipes poderiam abandonar a ferramenta interna e buscar alternativas externas se fosse muito difícil de usar). Hoje vemos várias empresas seguindo um rumo parecido – seja usando plataformas open-source como Kubeflow, seja construindo camadas customizadas –, mas sempre com a meta de reduzir atrito entre experimentação e produção via boas interfaces.

Caso 3: Plataformas low-code/no-code e o limite das abstrações

Nos últimos anos, outra tendência emergiu: ferramentas de AutoML ou plataformas low-code que prometem construir modelos sem que o usuário precise codificar. Embora não sejam exatamente “SDKs” (já que muitas vezes eliminam o código), elas oferecem APIs gráficas ou de alto nível. Exemplos incluem H2O.ai Driverless AI, Azure AutoML, entre outros. Essas ferramentas tiveram adoção em certas áreas para acelerar POCs e envolver não-programadores. Contudo, observamos também limitações: frequentemente, quando o problema foge um pouco

do padrão, o low-code não dá conta e os usuários precisam voltar para código e bibliotecas tradicionais. Assim, a tendência não deslocou a importância de SDKs como se pensava.

Ao contrário, muitas empresas usam AutoML para iniciar um projeto, mas depois migram para code/SDKs tradicionais ao refinar a solução. Isso ensina que a flexibilidade das APIs programáveis ainda é insuperável em muitos cenários complexos. As plataformas de AutoML, porém, influenciaram o design de APIs: bibliotecas como scikit-learn incorporaram funções de automação (como GridSearchCV para sintonizar parâmetros) de forma programática. Ou seja, mesmo modelos de alto nível acabam expondo SDKs para customização.

O equilíbrio atual no mercado parece ser: ferramentas de alto nível convivem com SDKs, cabendo à equipe escolher a ferramenta certa para cada etapa do projeto. Profissionais de pós-graduação em ML devem estar cientes dessas opções e, mais importante, entender que uma base sólida em programação e APIs de ML permite extrair o máximo de qualquer plataforma – seja usando diretamente ou integrando com camadas low-code.

Caso 4: Comunidade Open Source e padrões

Falando de tendências, não podemos ignorar o papel da comunidade open-source na formação de padrões de APIs. Por exemplo, o scikit-learn influenciou até bibliotecas em R e Julia sobre como estruturar interfaces de aprendizagem (no R surgiu o pacote caret e mais tarde o tidymodels, claramente inspirados em ideias do sklearn; em Python, até frameworks de deep learning adotaram o conceito de fit/evaluate em certa medida). Hoje, vemos a Hugging Face definindo padrões em NLP: modelos pré-treinados são disponibilizados via uma API consistente (from_pretrained etc.) e isso virou tão comum que outras bibliotecas fornecem compatibilidade com os objetos da Hugging Face.

Esse efeito em cascata significa que aprendermos uma API bem estabelecida nos habilita a usar muitas outras com facilidade, pois elas deliberadamente copiam elementos de design de quem acertou primeiro. A tendência, portanto, é uma convergência de interfaces: assim como GUI tem seus padrões UX, as APIs de ML estão convergindo para boas práticas quase universais (consistência, nomes claros,

objetos representando conceitos de ML etc.). Para profissionais, isso é bom – diminui o custo de aprender novas ferramentas. Para criadores(as) de ferramentas, há o desafio de equilibrar inovação com familiaridade: às vezes uma API nova precisa quebrar padrões para oferecer algo realmente diferente, mas se forçar o usuário a reaprender tudo do zero, pode fracassar.

Caso 5: APIs e responsabilidade – o desenvolvedor no centro

Por fim, uma tendência recente é pensar além da funcionalidade técnica: as APIs de ML também devem incorporar princípios de uso responsável e ético. Por exemplo, serviços de cloud ML agora incluem recursos para explicar modelos (explainability) e medição de viés e os SDKs expõem essas funcionalidades para que desenvolvedores(as) possam auditar seus modelos facilmente. O design de API aqui deve tornar fácil fazer a coisa certa. Se avaliar fairness de um modelo requer chamadas complicadíssimas, poucos farão; mas se estiver a um método de distância (model.explain() por exemplo), a adoção cresce. Portanto, espera-se que os futuros SDKs de ML integrem cada vez mais funcionalidades de transparência e governança e o desafio será fazê-lo sem complicar a experiência.

De modo geral, o panorama de mercado confirma: quem domina bons SDKs/APIs de ML tem vantagem competitiva, seja por produzir soluções mais rápido, seja por conseguir colaborar melhor (já que código claro atrai contribuições). Vivemos em um ecossistema em que novas bibliotecas surgem constantemente, mas as ideias de design circulam rapidamente entre elas. O(a) profissional de dados de hoje deve acompanhar essas tendências, experimentando ferramentas emergentes (por exemplo, a biblioteca JAX do Google, que segue uma filosofia de API funcional para alta performance, ou frameworks de MLOps como Metaflow da Netflix, que usa decoradores Python para facilitar pipelines).

Cada nova ferramenta que surge é uma chance de observar decisões de design diferentes. E, quem sabe, vocês mesmos(as) no futuro participarão da criação de uma biblioteca interna ou open-source – certamente esta disciplina os deixou mais preparados e preparadas para avaliar criticamente e aplicar princípios sólidos nesse trabalho.

Para saber mais sobre os casos mencionados: veja a [referência do BigGo](#) (2025) sobre a mudança de preferências de frameworks de IA; confira o [blog do Uber](#) sobre Michelangelo PyML; e explore artigos no Medium e podcasts do MLOps Community discutindo ferramentas de pipelines e AutoML.

EMAND

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos como SDKs e APIs bem projetadas podem simplificar radicalmente o desenvolvimento de pipelines de Machine Learning. Na introdução, entendemos o papel dos SDKs de ML como facilitadores que integram nosso código a plataformas robustas. Você viu, na prática, um exemplo de pipeline usando o scikit-learn e como a API unificada dele torna fácil compor transformações e modelos.

Em seguida, mergulhamos nos princípios de design de APIs: consistência, simplicidade, fluxos intuitivos, nomenclatura coesa, extensibilidade e feedback útil – todos fatores que tornam bibliotecas como scikit-learn e Keras referências de usabilidade. Analisamos também exemplos do mundo real, percebendo que ferramentas com melhor experiência de desenvolvimento tendem a prevalecer e que empresas investem em SDKs internos para ganhar velocidade e qualidade em seus projetos de ML.

Com isso, chegamos ao fim desta aula com várias lições-chave: escolher (ou projetar) bem as abstrações significa empoderar usuários sem esconder o necessário e invariavelmente leva a soluções mais elegantes e eficientes. Revisite os conceitos apresentados, consulte as referências fornecidas e, principalmente, pratique – aplique esses princípios em projetos do seu dia a dia. Assim, você consolidará o conhecimento e estará pronto(a) para enfrentar desafios maiores, construindo e usando APIs de ML de forma consciente e eficaz.

REFERÊNCIAS

BIGGO.COM. **The Great ML Framework Shift: How PyTorch Overtook TensorFlow in Research.** 2025. Disponível em: https://biggo.com/news/202510261933_PyTorch-TensorFlow-Research-Dominance-2025. Acesso em: 21 jan. 2026.

BUITINCK, L. et al. **API design for machine learning software:** experiences from the scikit-learn project. 2013. Disponível em: <https://inria.hal.science/hal-00856511/document>. Acesso em: 21 jan. 2026.

CHOLLET, F. **User experience design for APIs.** 2017. Disponível em: <https://blog.keras.io/user-experience-design-for-apis.html>. Acesso em: 21 jan. 2026.

GEEKSFORGEEEKS. **Pipeline API – Quickstart.** 2025. Disponível em: <https://www.geeksforgeeks.org/hugging-face-pipeline-abstraction/>. Acesso em: 21 jan. 2026.

RED HAT. **SDK – o que é e para que serve um SDK?** 2020. Disponível em: <https://www.redhat.com/pt-br/topics/cloud-native-apps/what-is-SDK>. Acesso em: 21 jan. 2026.

TOWARDSDATASCIENCE.COM. **What is Kubeflow Pipelines?** 2021. Disponível em: <https://towardsdatascience.com/kubeflow-pipelines-how-to-build-your-first-kubeflow-pipeline-from-scratch-2424227f7e5/>. Acesso em: 21 jan. 2026.

UBER.COM. **Michelangelo PyML:** Introducing Uber's Platform for Rapid Python ML Model Development. 2018. Disponível em: <https://www.uber.com/blog/michelangelo-pym/>. Acesso em: 21 jan. 2026.

PALAVRAS-CHAVE

SDK. Design de API. Pipeline de Machine Learning.

EMENDAS



POSTECH